Class 08: Unsupervised Learning Mini-Project

Hyuntae An (TSN:200231511)

Background

The goal of this mini-project is for you to explore a complete analysis using the machine learning techniques covered in class.

Today we will analyze a bippsy data-set from fine needle aspiration (FNA) of breast mass.

Data Import

The data is made available as a CSV filr for download. We can read this with the `read.csv`

```
# Save your input data file into your Project directory
fna.data <- "WisconsinCancer.csv"

# Complete the following code to input the data and store as wisc.df
wisc.df <- read.csv(fna.data, row.names=1)
```

```
head(wisc.df, 4)
```

```
         diagnosis radius_mean texture_mean perimeter_mean area_mean
842302           M       17.99        10.38         122.80    1001.0
842517           M       20.57        17.77         132.90    1326.0
84300903         M       19.69        21.25         130.00    1203.0
84348301         M       11.42        20.38          77.58     386.1
         smoothness_mean compactness_mean concavity_mean concave.points_mean
842302           0.11840          0.27760         0.3001             0.14710
842517           0.08474          0.07864         0.0869             0.07017
84300903         0.10960          0.15990         0.1974             0.12790
84348301         0.14250          0.28390         0.2414             0.10520
         symmetry_mean fractal_dimension_mean radius_se texture_se perimeter_se
842302          0.2419                0.07871    1.0950     0.9053        8.589
```

```
842517          0.1812                0.05667    0.5435     0.7339        3.398
84300903        0.2069                0.05999    0.7456     0.7869        4.585
84348301        0.2597                0.09744    0.4956     1.1560        3.445
          area_se smoothness_se compactness_se concavity_se concave.points_se
842302     153.40      0.006399        0.04904      0.05373           0.01587
842517      74.08      0.005225        0.01308      0.01860           0.01340
84300903    94.03      0.006150        0.04006      0.03832           0.02058
84348301    27.23      0.009110        0.07458      0.05661           0.01867
          symmetry_se fractal_dimension_se radius_worst texture_worst
842302        0.03003             0.006193        25.38         17.33
842517        0.01389             0.003532        24.99         23.41
84300903      0.02250             0.004571        23.57         25.53
84348301      0.05963             0.009208        14.91         26.50
          perimeter_worst area_worst smoothness_worst compactness_worst
842302             184.60     2019.0           0.1622            0.6656
842517             158.80     1956.0           0.1238            0.1866
84300903           152.50     1709.0           0.1444            0.4245
84348301            98.87      567.7           0.2098            0.8663
          concavity_worst concave.points_worst symmetry_worst
842302             0.7119               0.2654         0.4601
842517             0.2416               0.1860         0.2750
84300903           0.4504               0.2430         0.3613
84348301           0.6869               0.2575         0.6638
          fractal_dimension_worst
842302                    0.11890
842517                    0.08902
84300903                  0.08758
84348301                  0.17300
```

To make sure we don't accidentally include this in our analysis, let's create a new data.frame that omits this first column:

```
# We can use -1 here to remove the first column
wisc.data <- wisc.df[,-1]
```

Finally, setup a separate new vector called diagnosis that contains the data from the diagnosis column of the original dataset. We will store this as a factor (useful for plotting) and use this later to check our results.

```
# Create diagnosis vector for later
diagnosis <- wisc.df$diagnosis
```

Exploratory Analysis

Before diving into PCA or clustering, take a few minutes to understand the structure of your data—how many samples, how many features, and whether there are any obvious issues like missing values or unexpected column types.

Explore the data you created before (`wisc.data` and `diagnosis`) to answer the following questions:

Q1. How many observations are in this dataset?

```
nrow(wisc.data)
```

```
[1] 569
```

There are 569 observations.

Q2. How many of the observations have a malignant diagnosis?

```
table(diagnosis)
```

```
diagnosis
  B   M
357 212
```

212 are malignant

Q3. How many variables/features in the data are suffixed with `_mean`?

```
length(grep("_mean", colnames(wisc.data)))
```

```
[1] 10
```

There are 10 features that end in `_mean`

Principal Component Analysis

Performing PCA

The next step in your analysis is to perform principal component analysis (PCA) on wisc.data.

It is important to check if the data need to be scaled before performing PCA. Recall two common reasons for scaling data include:

- The input variables use different units of measurement.
- The input variables have significantly different variances.

Check the mean and standard deviation of the features (i.e. columns) of the `wisc.data` to determine if the data should be scaled. Use the `colMeans()` and `apply()` functions like you've done before.

```
# Check column means and standard deviations
colMeans(wisc.data)
```

```
            radius_mean            texture_mean          perimeter_mean
           1.412729e+01            1.928965e+01            9.196903e+01
              area_mean         smoothness_mean        compactness_mean
           6.548891e+02            9.636028e-02            1.043410e-01
         concavity_mean     concave.points_mean          symmetry_mean
           8.879932e-02            4.891915e-02            1.811619e-01
 fractal_dimension_mean               radius_se              texture_se
           6.279761e-02            4.051721e-01            1.216853e+00
            perimeter_se                 area_se          smoothness_se
           2.866059e+00            4.033708e+01            7.040979e-03
         compactness_se            concavity_se       concave.points_se
           2.547814e-02            3.189372e-02            1.179614e-02
             symmetry_se    fractal_dimension_se            radius_worst
           2.054230e-02            3.794904e-03            1.626919e+01
          texture_worst         perimeter_worst              area_worst
           2.567722e+01            1.072612e+02            8.805831e+02
       smoothness_worst       compactness_worst         concavity_worst
           1.323686e-01            2.542650e-01            2.721885e-01
   concave.points_worst          symmetry_worst fractal_dimension_worst
           1.146062e-01            2.900756e-01            8.394582e-02
```

```
apply(wisc.data,2,sd)
```

```
          radius_mean           texture_mean         perimeter_mean
         3.524049e+00           4.301036e+00           2.429898e+01
            area_mean        smoothness_mean       compactness_mean
         3.519141e+02           1.406413e-02           5.281276e-02
       concavity_mean    concave.points_mean          symmetry_mean
         7.971981e-02           3.880284e-02           2.741428e-02
fractal_dimension_mean              radius_se             texture_se
         7.060363e-03           2.773127e-01           5.516484e-01
         perimeter_se                area_se         smoothness_se
         2.021855e+00           4.549101e+01           3.002518e-03
       compactness_se           concavity_se      concave.points_se
         1.790818e-02           3.018606e-02           6.170285e-03
          symmetry_se   fractal_dimension_se            radius_worst
         8.266372e-03           2.646071e-03           4.833242e+00
        texture_worst        perimeter_worst              area_worst
         6.146258e+00           3.360254e+01           5.693570e+02
     smoothness_worst      compactness_worst        concavity_worst
         2.283243e-02           1.573365e-01           2.086243e-01
  concave.points_worst         symmetry_worst fractal_dimension_worst
         6.573234e-02           6.186747e-02           1.806127e-02
```

```
# Perform PCA on wisc.data by completing the following code
wisc.pr <- prcomp(wisc.data)
summary(wisc.pr)
```

```
Importance of components:
                          PC1      PC2      PC3     PC4     PC5     PC6   PC7
Standard deviation     666.170 85.49912 26.52987 7.39248 6.31585 1.73337 1.347
Proportion of Variance   0.982  0.01618  0.00156 0.00012 0.00009 0.00001 0.000
Cumulative Proportion    0.982  0.99822  0.99978 0.99990 0.99999 0.99999 1.000
                           PC8    PC9   PC10   PC11    PC12    PC13    PC14
Standard deviation      0.6095 0.3944 0.2899 0.1778 0.08659 0.05623 0.04649
Proportion of Variance  0.0000 0.0000 0.0000 0.0000 0.00000 0.00000 0.00000
Cumulative Proportion   1.0000 1.0000 1.0000 1.0000 1.00000 1.00000 1.00000
                          PC15   PC16    PC17    PC18    PC19    PC20     PC21
Standard deviation     0.03642 0.0253 0.01936 0.01534 0.01359 0.01281 0.008838
Proportion of Variance 0.00000 0.0000 0.00000 0.00000 0.00000 0.00000 0.000000
Cumulative Proportion  1.00000 1.0000 1.00000 1.00000 1.00000 1.00000 1.000000
                          PC22     PC23     PC24     PC25     PC26     PC27
Standard deviation     0.00759 0.005909 0.005329 0.004018 0.003534 0.001918
Proportion of Variance 0.00000 0.000000 0.000000 0.000000 0.000000 0.000000
Cumulative Proportion  1.00000 1.000000 1.000000 1.000000 1.000000 1.000000
```

```
                            PC28     PC29      PC30
Standard deviation     0.001688 0.001416 0.0008379
Proportion of Variance 0.000000 0.000000 0.0000000
Cumulative Proportion  1.000000 1.000000 1.0000000
```

```
wisc.pr <- prcomp(wisc.data, scale = TRUE)
pca.var <- summary(wisc.pr)$importance
```

> Q4. From your results, what proportion of the original variance is captured by the first principal component (PC1)?

```
pca.var["Proportion of Variance", "PC1"]
```

```
[1] 0.44272
```

PC1 captures 0.4427, about 44.3% of the variance.

> Q5. How many principal components (PCs) are required to describe at least 70% of the original variance in the data?

```
which(pca.var["Cumulative Proportion", ] >= 0.70)[1]
```

```
PC3
  3
```

3 PCs are needed to reach 70% (cumulative 72.6%).

> Q6. How many principal components (PCs) are required to describe at least 90% of the original variance in the data?

```
which(pca.var["Cumulative Proportion", ] >= 0.90)[1]
```

```
PC7
  7
```

7 PCs are needed to reach 90% (cumulative 91.0%).

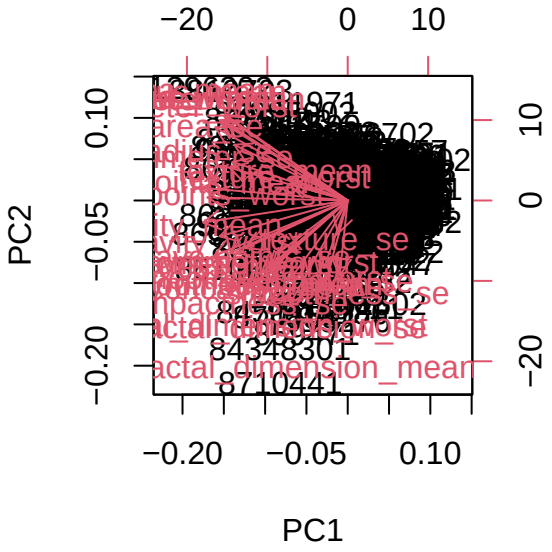
Interpreting PCA results

Now you will use some visualizations to better understand your PCA model. A common visualization for PCA results is the so-called biplot.

However, you will often run into some common challenges with using biplots on real-world data containing a non-trivial number of observations and variables. Here we will need to look at some alternative visualizations. You are encouraged to experiment with additional visualizations before moving on to the next section.

Create a biplot of the wisc.pr using the `biplot()` function.

```
biplot(wisc.pr)
```



Q7. What stands out to you about this plot? Is it easy or difficult to understand? Why?

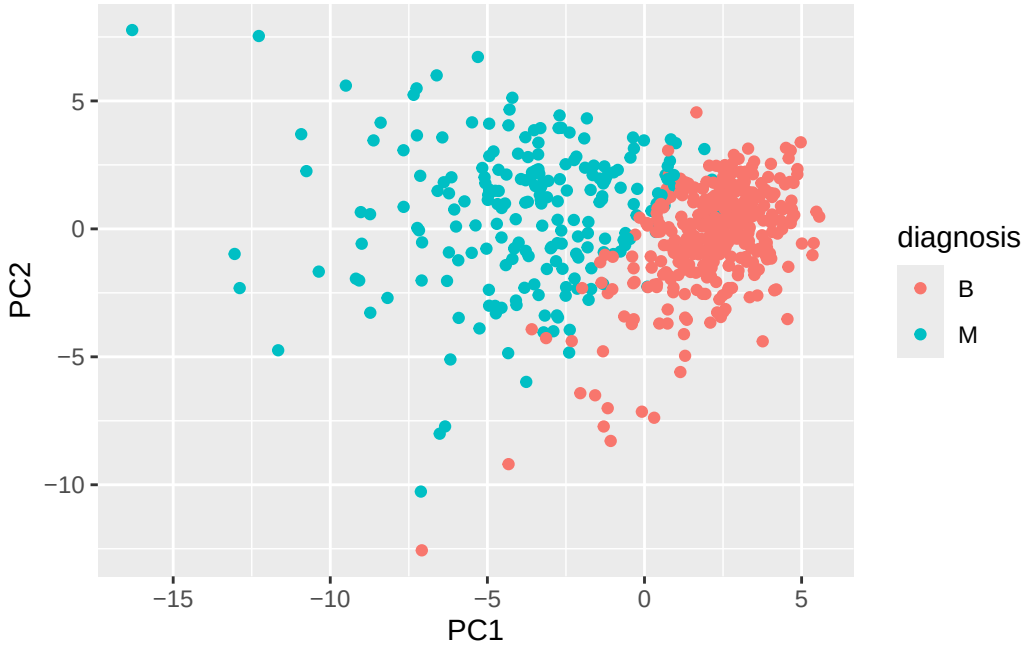
The biplot is very hard to read. All 569 sample row names and 30 arrows are drawn on top of each other, so it's a crowded mess, and you can't see any grouping or trend. You need a simpler scatter plot of the PC scores colored by diagnosis.

Rownames are used as the plotting character for biplots like this one which can make trends rather hard to see. In fact, this plot is very poor. So lets generate a more standard scatter plot of each observation along principal components 1 and 2 (i.e. a plot of PC1 vs PC2 available as

the first two columns of `wisc.pr$x`) and color the points by the `diagnosis` (available in the diagnosis vector you created earlier).

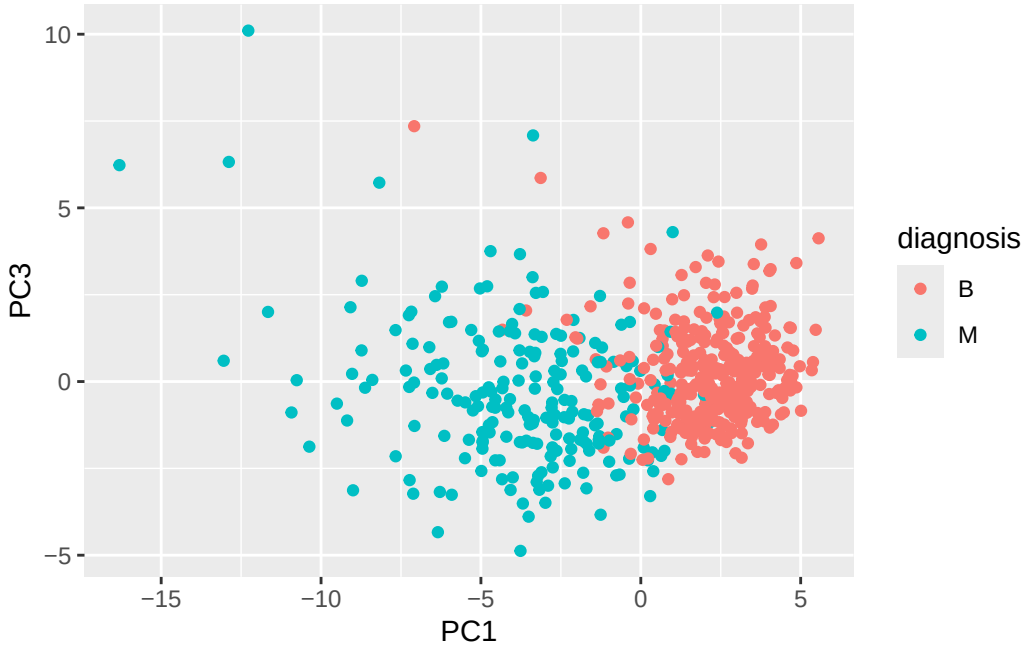
```
# Scatter plot observations by components 1 and 2
library(ggplot2)

ggplot(wisc.pr$x) +
  aes(PC1, PC2, col = diagnosis) +
  geom_point()
```



Q8. Generate a similar plot for principal components 1 and 3. What do you notice about these plots?

```
# Repeat for components 1 and 3
ggplot(wisc.pr$x) +
  aes(PC1, PC3, col = diagnosis) +
  geom_point()
```



Variance Explained

A scree plot shows how much variance each PC captures. We typically look for an "elbow" — a point where adding more PCs gives diminishing returns. This can help us decide how many PCs to consider for further analysis. (Spoiler: some real data sets don't have a perfect elbow, so folks will often use a threshold like 70% or 90% cumulative variance instead.)

Calculate the variance of each principal component by squaring the `sdev` component of `wisc.pr` (i.e. `wisc.pr$sdev^2`). Save the result as an object called `pr.var`.

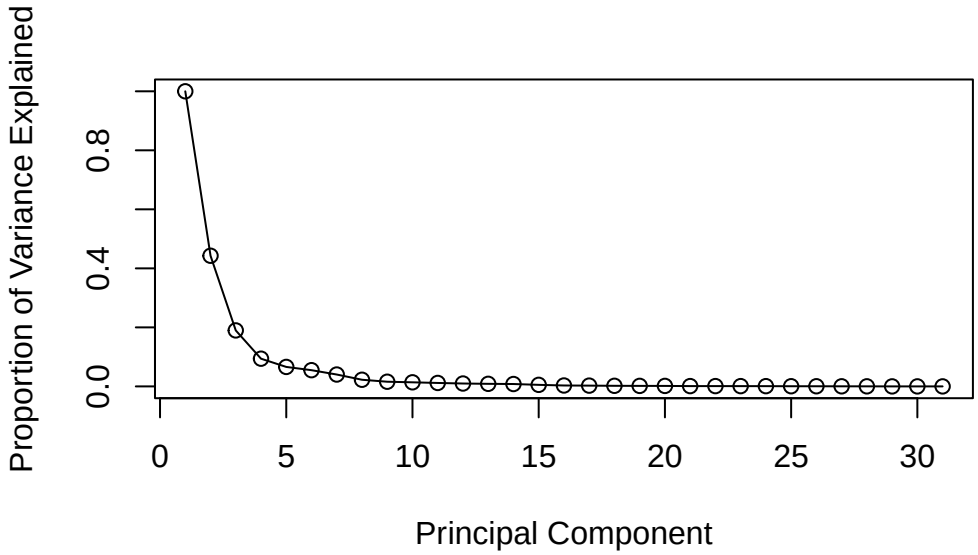
```
# Calculate variance of each component
pr.var <- wisc.pr$sdev^2
head(pr.var)
```

```
[1] 13.281608  5.691355  2.817949  1.980640  1.648731  1.207357
```

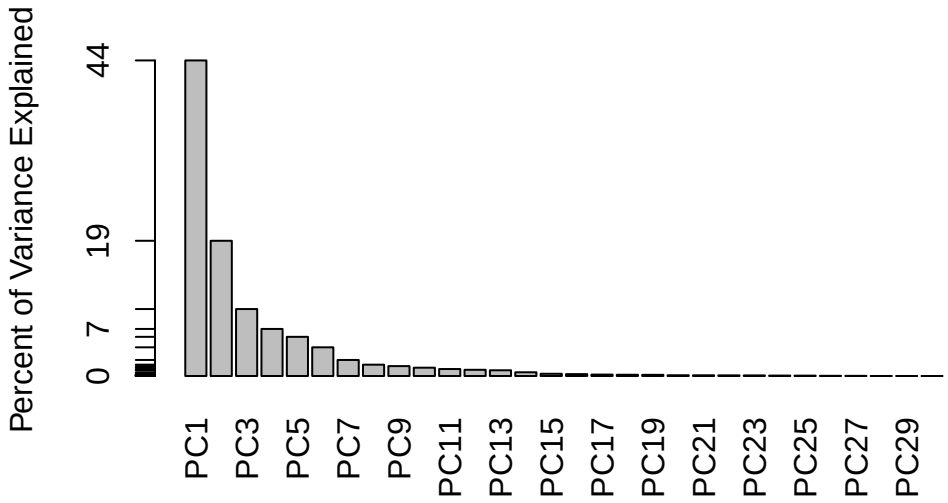
Calculate the variance explained by each principal component by dividing by the total variance explained of all principal components. Assign this to a variable called `pve` and create a plot of variance explained for each principal component.

```
# Variance explained by each principal component: pve
pve <- pr.var / sum(pr.var)

# Plot variance explained for each principal component
plot(c(1,pve), xlab = "Principal Component",
     ylab = "Proportion of Variance Explained",
     ylim = c(0, 1), type = "o")
```




```
# Alternative scree plot of the same data, note data driven y-axis
barplot(pve, ylab = "Percent of Variance Explained",
     names.arg=paste0("PC",1:length(pve)), las=2, axes = FALSE)
axis(2, at=pve, labels=round(pve,2)*100 )
```



Communicating PCA results

In this section we will check your understanding of the PCA results, in particular the "loadings" and "variance explained".

The loading vector (`wisc.pr$rotation`) tells us which original measurements contribute most to each PC.

A large PC1 loading value (positive or negative) for one of the 30 original measurements (i.e. features/columns we started with), for example, would suggest that this feature is an important driver of the variation we see in the score plot and thus helpful for distinguishing "M" from "B" samples. Let's check which features matter most to PC1.

> Q9. For the first principal component, what is the component of the loading vector (i.e. `wisc.pr$rotation[,1]`) for the feature `concave.points_mean`? This tells us how much this original feature contributes to the first PC. Are there any features with larger contributions than this one?

```
# PC1 loading for concave.points_mean
wisc.pr$rotation["concave.points_mean", 1]
```

```
[1] -0.2608538
```

```
# Are any features larger? Sort PC1 loadings by absolute size
head(sort(abs(wisc.pr$rotation[, 1]), decreasing = TRUE))
```

```
 concave.points_mean       concavity_mean concave.points_worst
           0.2608538            0.2584005            0.2508860
    compactness_mean      perimeter_worst      concavity_worst
           0.2392854            0.2366397            0.2287675
```

The PC1 loading for concave.points_mean is −0.2609. It is the largest contribution in absolute value, so no feature contributes more. The next largest are concavity_mean (−0.258) and concave.points_worst (−0.251).

Hierarchical clustering

The goal of this section is to do hierarchical clustering of the original datato see if there is any obvious grouping into malignant and benign clusters.

Recall from class that hierarchical clustering does not assume in advance the number of natural groups that exist in the data (unlike K-means clustering).

As part of the preparation for hierarchical clustering, the distance between all pairs of observations needs to be calculated. This "distance matrix" will be the input for the `hclust()` function.

One of the optional arguments to `hclust()` allows you to pick different ways (a.k.a. "methods") to link clusters together, with **single**, **complete**, and **average** being the most common "linkage methods". You can explore the effectes of these different methods in this section.

First scale the `wisc.data` data and assign the result to `data.scaled`.

```
# Scale the wisc.data data using the "scale()" function
data.scaled <- scale(wisc.data)
```

Next, calculate the (Euclidean) distances between all pairs of observations in the new scaled dataset and assign the result to `data.dist`.

```
data.dist <- dist(data.scaled)
```

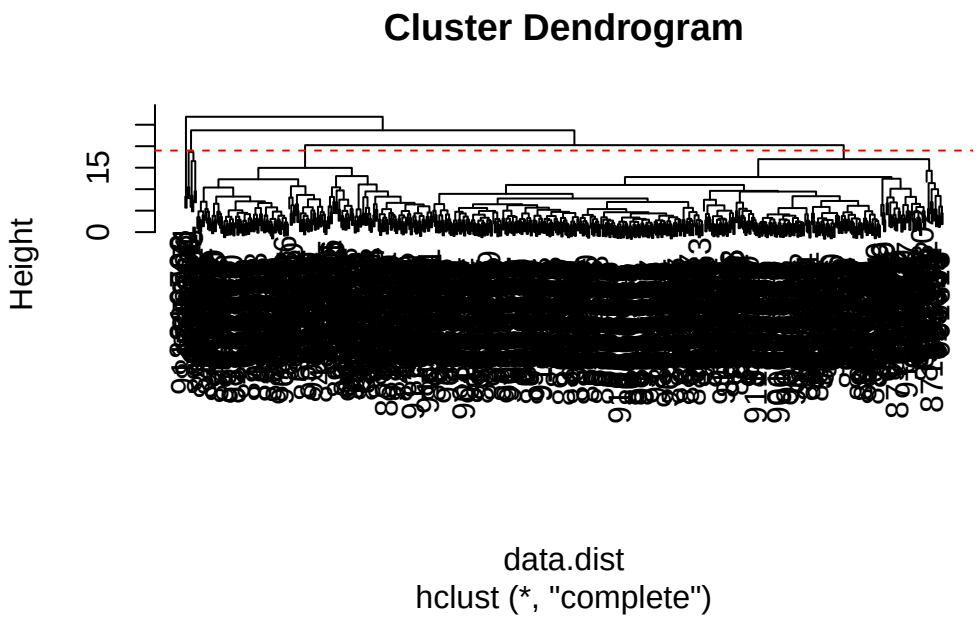
Create a hierarchical clustering model using complete linkage. Manually specify the method argument to hclust() and assign the results to `wisc.hclust`.

```
wisc.hclust <- hclust(data.dist, method = "complete")
```

Results of hierarchical clustering

Q10. Using the plot() and abline() functions, what is the height at which the clustering model has 4 clusters?

```
plot(wisc.hclust)
abline(h = 19, col = "red", lty = 2)
```



the tree has 4 clusters at a height of about 19. The red dashed line at h = 19 crosses exactly 4 vertical branches, matching the dendrogram on the lab page.

Selecting number of clusters

Normally in unsupervised learning we don't have labels to check our work against. Here we're lucky—the pathologist's diagnosis lets us evaluate whether our clusters correspond to something biologically meaningful. Let's see how well the clustering separates malignant from benign samples.

Use `cutree()` to cut the tree so that it has 4 clusters. Assign the output to the variable `wisc.hclust.clusters`.

```
wisc.hclust.clusters <- cutree(wisc.hclust, k = 4)
```

We can use the `table()` function to compare the cluster membership to the actual diagnoses.

```
table(wisc.hclust.clusters, diagnosis)
```

```
                    diagnosis
wisc.hclust.clusters   B   M
                   1  12 165
                   2   2   5
                   3 343  40
                   4   0   2
```

Here we picked four clusters and see that cluster 1 largely corresponds to malignant cells (with d**iagnosis** values of "M") whilst cluster 3 largely corresponds to benign cells (with **diagnosis** values of "B").

Using different methods

As we discussed in our last class videos there are number of different "methods" we can use to combine points during the hierarchical clustering procedure. These include "single", "complete", "average" and (my favorite) "ward.D2".

> Q12. Which method gives your favorite results for the same data.dist dataset? Explain your reasoning.

```
# Q12. Compare linkage methods on the same data.dist
for (m in c("single", "complete", "average", "ward.D2")) {
  hc <- hclust(data.dist, method = m)
  cat("\nMethod:", m, "\n")
  print(table(cutree(hc, k = 2), diagnosis))
}
```

```
Method: single
   diagnosis
      B   M
  1 357 210
  2   0   2
```

```
Method: complete
   diagnosis
      B   M
  1 357 210
  2   0   2

Method: average
   diagnosis
      B   M
  1 357 209
  2   0   3

Method: ward.D2
   diagnosis
      B   M
  1  20 164
  2 337  48
```

`ward.D2` gives the best results. With 2 clusters, single, complete and average put almost all samples into one cluster, while ward.D2 splits them into two balanced groups that mostly match the malignant and benign diagnoses. It does this by minimizing variance within each cluster.

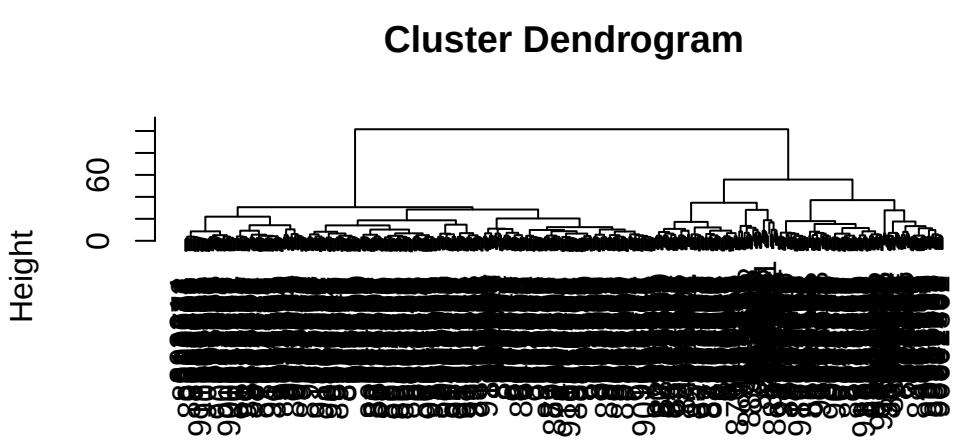
Combining Methods

###Clustering on PCA results

Using the minimum number of principal components required to describe at least 90% of the variability in the data, create a hierarchical clustering model with the linkage `method="ward.D2"`. We use Ward's criterion here because it is based on multidimensional variance like principal components analysis. Assign the results to `wisc.pr.hclust`.

```r
# Cluster on the first 7 PCs (at least 90% of the variance) with Ward's method
wisc.pr.hclust <- hclust(dist(wisc.pr$x[, 1:7]), method = "ward.D2")

# Plot the dendrogram
plot(wisc.pr.hclust)
```




```
grps <- cutree(wisc.pr.hclust, k = 2)
table(grps)
```

```
grps
  1   2
216 353
```

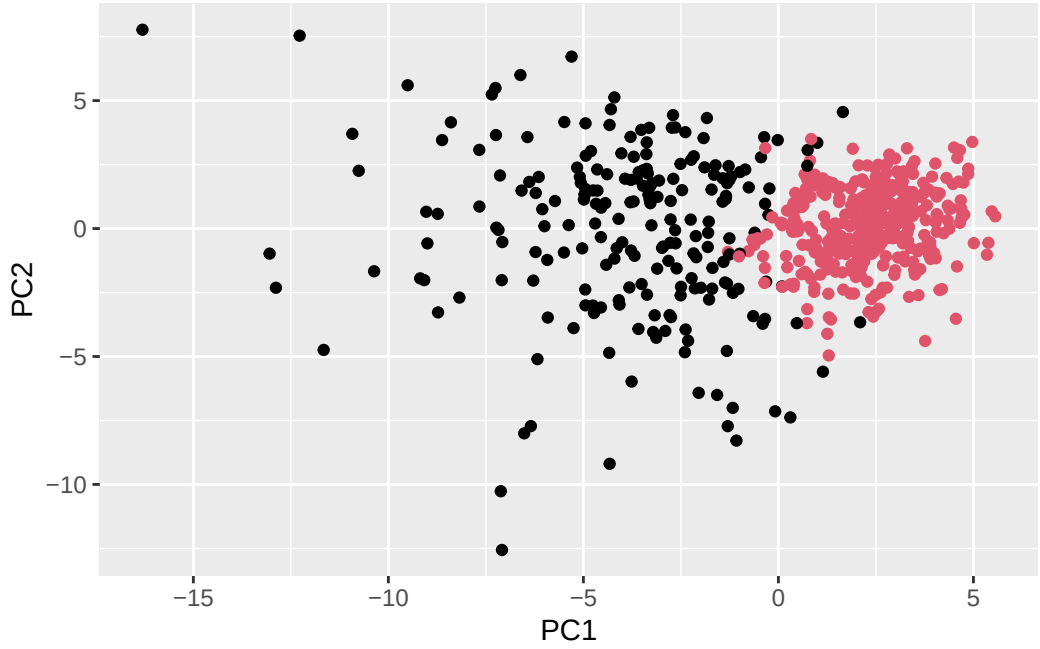
```
grps <- cutree(wisc.pr.hclust, k=2)
table(grps)
```

```
grps
  1   2
216 353
```

```
table(grps, diagnosis)
```

```
    diagnosis
grps   B   M
   1  28 188
   2 329  24
```

```
ggplot(wisc.pr$x) +
  aes(PC1, PC2) +
  geom_point(col=grps)
```




```
# Cluster on the first 7 PCs with Ward's method
wisc.pr.hclust <- hclust(dist(wisc.pr$x[, 1:7]), method = "ward.D2")

wisc.pr.hclust.clusters <- cutree(wisc.pr.hclust, k = 2)
```

Q13. How well does the newly created hclust model with two clusters separate out the two "M" and "B" diagnoses?

```
# Compare to actual diagnoses
table(wisc.pr.hclust.clusters, diagnosis)
```

```
                     diagnosis
wisc.pr.hclust.clusters   B   M
                      1  28 188
                      2 329  24
```

It separates the diagnoses well. Cluster 1 is mostly malignant (188 M, 28 B) and cluster 2 is mostly benign (329 B, 24 M), so only 52 of 569 samples are in the wrong cluster.

Q14. How well do the hierarchical clustering models you created in the previous sections (i.e. without first doing PCA) do in terms of separating the diagnoses? Again, use the `table()` function to compare the output of each model (`wisc.hclust.clusters` and `wisc.pr.hclust.clusters`) with the vector containing the actual diagnoses.

```
table(wisc.hclust.clusters, diagnosis)
```

```
                    diagnosis
wisc.hclust.clusters   B   M
                   1  12 165
                   2   2   5
                   3 343  40
                   4   0   2
```

The model without PCA does worse. It needs 4 clusters, two of which are tiny and uninformative, and its benign cluster still contains 40 malignant samples. Running PCA first gives a cleaner two-group split.

Prediction

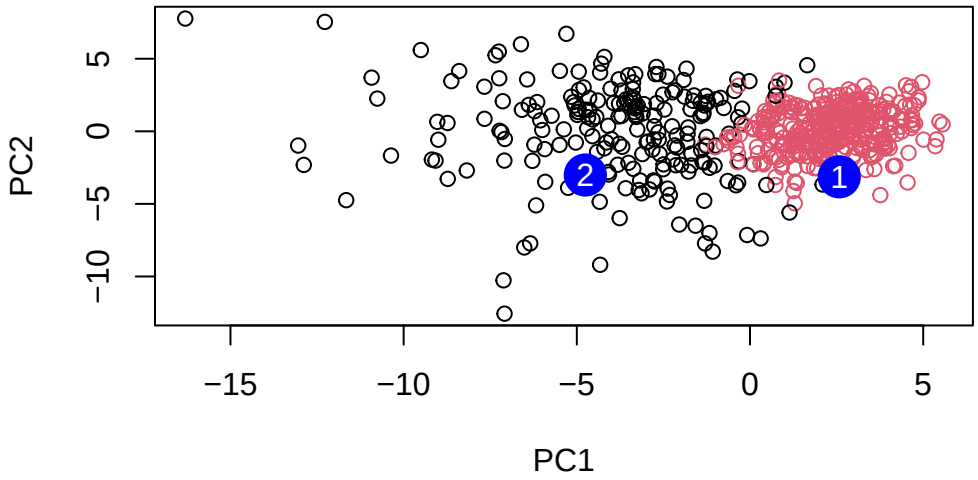
We will use the `predict()` function that will take our PCA model from before and new cancer cell data and project that data onto our PCA space.

```
#url <- "new_samples.csv"
url <- "https://tinyurl.com/new-samples-CSV"
new <- read.csv(url)
npc <- predict(wisc.pr, newdata=new)
npc
```

```
          PC1       PC2        PC3        PC4       PC5        PC6        PC7
[1,]  2.576616 -3.135913  1.3990492 -0.7631950  2.781648 -0.8150185 -0.3959098
[2,] -4.754928 -3.009033 -0.1660946 -0.6052952 -1.140698 -1.2189945  0.8193031
           PC8       PC9       PC10       PC11      PC12      PC13     PC14
[1,] -0.2307350 0.1029569 -0.9272861 0.3411457  0.375921 0.1610764 1.187882
[2,] -0.3307423 0.5281896 -0.4855301 0.7173233 -1.185917 0.5893856 0.303029
         PC15       PC16        PC17        PC18        PC19       PC20
[1,] 0.3216974 -0.1743616 -0.07875393 -0.11207028 -0.08802955 -0.2495216
[2,] 0.1299153  0.1448061 -0.40509706  0.06565549  0.25591230 -0.4289500
          PC21       PC22       PC23       PC24        PC25         PC26
```

```
[1,]  0.1228233 0.09358453 0.08347651  0.1223396  0.02124121  0.078884581
[2,] -0.1224776 0.01732146 0.06316631 -0.2338618 -0.20755948 -0.009833238
              PC27        PC28          PC29         PC30
[1,]  0.220199544 -0.02946023 -0.015620933  0.005269029
[2,] -0.001134152  0.09638361  0.002795349 -0.019015820
```

```
plot(wisc.pr$x[,1:2], col=grps)
points(npc[,1], npc[,2], col="blue", pch=16, cex=3)
text(npc[,1], npc[,2], c(1,2), col="white")
```



Q16. Which of these new patients should we prioritize for follow up based on your results?

We should prioritize patient 2. In the PC1 vs PC2 plot, patient 2 (PC1 −4.75) falls among the malignant samples, while patient 1 (PC1 +2.58) falls among the benign ones.

Summary

In this mini-project, you applied unsupervised learning techniques to real-world cancer cell data from fine needle aspiration biopsies. You learned how to:

- Apply PCA to reduce dimensionality while retaining meaningful variance

- Interpret PCA results through scree plots, score plots and loading vectors (the 3 main result figures from PCA)
- Perform hierarchical clustering with different linkage methods
- Combine PCA with clustering for improved separation of sample groups
- Evaluate clustering performance using sensitivity and specificity metrics
- Use your trained PCA model to make predictions on new patient samples

A key takeaway is that preprocessing with PCA before clustering often yields better results than clustering on raw data alone. This workflow (dimensionality reduction followed by clustering) is widely used in biomedical research for tasks like patient stratification, biomarker discovery, and disease subtyping.

About this document

Here we use the `sessionInfo()` function to report on our R systems setup at the time of document execution.

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.6.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Los_Angeles
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] ggplot2_4.0.3

loaded via a namespace (and not attached):
 [1] vctrs_0.7.3        cli_3.6.6          knitr_1.52         rlang_1.3.0
 [5] xfun_0.61          otel_0.2.0         generics_0.1.4     S7_0.2.2
```

```
 [9] jsonlite_2.0.0     labeling_0.4.3     glue_1.8.1          htmltools_0.5.9
[13] scales_1.4.0       rmarkdown_2.32     grid_4.6.1          tibble_3.3.1
[17] evaluate_1.0.5     fastmap_1.2.0      yaml_2.3.12         lifecycle_1.0.5
[21] compiler_4.6.1     dplyr_1.2.1        RColorBrewer_1.1-3  pkgconfig_2.0.3
[25] rstudioapi_0.19.0  farver_2.1.2       digest_0.6.39       R6_2.6.1
[29] tidyselect_1.2.1   pillar_1.11.1      magrittr_2.0.5      withr_3.0.3
[33] tools_4.6.1        gtable_0.3.6
```